Министерство образования Республики Беларусь

Учреждение образования

Белорусский Государственный университет информатики и радиоэлектроники

Инженерно-экономический факультет

Кафедра экономической информатики

Отчет

по лабораторной работе № 4

на тему «Разработка бизнес-операций и обработки данных в GraphQL API»

Выполнила: Абу Халава И.М.
студент группы 377901

Проверил: Сильванович Ю.В.

Минск 2026

Введение

Лабораторная работа № 4 посвящена реализации бизнес-операций, фильтрации и агрегации GraphQL API в программном средстве анализа рентабельности продукции. Система объединяет производственный и финансовый учёт, рассчитывает себестоимость и прибыль, предоставляет GraphQL API и отчётность. Исправления бизнес-логики и инфраструктуры собраны в итоговой исполняемой версии lab7_profitability_system; прежние каталоги лабораторных сохраняют историю этапов разработки.

Сервисы владеют отдельными базами данных. Gateway проверяет пользователя и права, сервисный слой выполняет бизнес-операции, репозитории сохраняют данные только своего сервиса. В итоговой версии синхронный обмен выполняется по gRPC, а асинхронные реакции — через Kafka.

1 Бизнес-операции GraphQL

Mutation	Результат и проверки
startProductionImport	ERP/MES, роль PRODUCTION_ACCOUNTANT; загрузка заказов, материалов и труда
startFinancialImport	FINANCE, роль FINANCE_ACCOUNTANT; загрузка реализации, накладных расходов и корректировок
retryImport	Повтор только FAILED, полномочия соответствующего контура; факты не дублируются
resolveExternalMapping	Существующий объект Master Data, допустимый тип и внешний код
calculateCost	Новая версия калькуляции, сохранённые накладные распределения и версии источников
calculateProfitability	Расчёт стоимости продаж, прибыли и маржи; CalculationJob COMPLETED
exportReport	Готовый CSV/XLSX/PDF с file_ref либо FAILED с описанием ошибки

2 Реальная загрузка производственных и финансовых записей

Import adapter читает JSON-массив из IMPORT_DATA_DIR/<SOURCE>/<YYYY-MM-01>.json. В Compose каталог подключён как /imports; демонстрационные снимки находятся в examples/imports. Отсутствующий файл, пустой массив и неверный формат фиксируются как ошибки загрузки, а не как успешный импорт нуля записей.

Производственная запись содержит external_id, product_code, quantity, materials и labor. Для каждого внешнего кода запрашивается LookupExternalMapping. В локальной транзакции сохраняются PRODUCTION_ORDER, MATERIAL_CONSUMPTION и LABOR_ENTRY. Ошибка любой вложенной записи откатывает всю запись заказа. Финансовая запись SALE сохраняет SALES_DOCUMENT и SALES_ITEM; OVERHEAD/CORRECTION сохраняются в FINANCIAL_ENTRY с исходным cost_center_id.

```
# Производственный снимок
[{
  "external_id": "OCT-ORDER-1", "product_code": "PD-1", "quantity": "10",
  "materials": [{"material_code": "PCB-01", "quantity": "10", "amount": "185"}],
  "labor": [{"cost_center_code": "ASSEMBLY", "hours": "5", "amount": "150"}]
}]
```

IMPORT_RUN создаётся в RUNNING. По завершении фиксируются records_loaded, records_failed и finished_at. При ошибках состояние FAILED, для каждой ошибки сохраняются record_key,

error_code, message и raw_payload. Если все записи сохранены, состояние COMPLETED. Событие результата сохраняется в outbox вместе с итогом загрузки; успешное событие соответствует реально принятому снимку.

3 Повтор импорта

retryImport очищает прежние ошибки и счётчики, возвращает запуск в RUNNING и повторно выполняет adapter по тому же источнику и месяцу. UUID фактов стабильно вычисляется из источника, типа и external_id. Повтор обновляет существующую запись и заменяет её дочерние факты, поэтому после исправления сопоставления не возникают дубликаты. Внешние данные должны представлять повторяемый снимок записей, а не неидентифицированные приращения.

4 Фильтрация и сводные показатели

products поддерживает группу, активность, строку поиска и сортировку по коду/названию. profitability поддерживает productIds, groupIds, costCenterIds, период, минимальную маржу и сортировку по периоду, выручке, прибыли и марже. Фильтры пересекаются. Отсутствующий productIds не ограничивает выборку, но пустой [] обязательно даёт пустой результат. Пустая группа также даёт [], а profitabilitySummary возвращает count=0 и нулевые суммы.

В итоговой версии Gateway получает аналитику из Reporting read model. Для группы фасад получает продукты Master Data; для центра затрат — продукты, участвующие в распределении Costing. Агрегация использует ту же отфильтрованную выборку. averageMarginPercent — среднее арифметическое маржинальности строк, а не отношение общей прибыли к общей выручке; значения этих двух метрик могут отличаться.

```
type ProfitabilitySummary {
  count: Int!
  totalRevenue: Decimal!
  totalCost: Decimal!
  totalProfit: Decimal!
  averageMarginPercent: Decimal!
}
```

5 Расчёт и сохранение накладных расходов

Costing получает выпуск и затраты вместе с allocation_lines. Каждая строка содержит production_order_id, cost_center_id и значения баз volume, labor_hours, direct_cost. Финансовый сервис возвращает сумму исходных накладных расходов и корректировок по центру затрат. Распределение выполняется внутри центра по выбранной базе; последний получатель получает остаток округления до копеек.

Одной транзакцией сохраняются COST_CALCULATION, все OVERHEAD_ALLOCATION и событие в outbox. Строка распределения содержит calculation_id, production_order_id, cost_center_id, basis_type, basis_value и amount. Сумма распределений равна overhead_cost калькуляции, сумма по центрам равна исходным финансовым суммам. Ненулевая сумма без положительной базы отклоняется до сохранения результатов.

```
total_cost = material_cost + labor_cost + overhead_cost
unit_cost = total_cost / produced_qty
unit_cost_deviation = unit_cost - planned_unit_cost
cost_of_sales = unit_cost * sold_qty
profit = revenue - cost_of_sales
margin_percent = profit / revenue * 100
```

Утверждённый план хранится в Master Data, его версия и значение фиксируются в калькуляции. При отсутствии плана plannedUnitCost и unitCostDeviation равны null. Query overheadAllocations раскрывает распределения по заказу и центру затрат, что позволяет проверить итоговую сумму, а

не только видеть overhead_cost.

6 Формирование файлов отчёта

До создания REPORT_EXPORT сервис проверяет существование SavedReport и совпадение его user_id с requested_by. requested_by определяется из проверенного Actor в Gateway. Затем применяются параметры сохранённого отчёта и читается витрина Reporting. CSV формируется с UTF-8 BOM, XLSX — как рабочая книга, PDF — как таблица с повторяемой строкой заголовка.

Файл сначала записывается во временный путь, затем атомарно переименовывается в каталог exports. Только после этого сохраняются file_ref, completed_at и READY. При ошибке формируется FAILED с error_message и без ссылки на якобы готовый файл. Получение статуса чужой выгрузки запрещено фасадом.

7 Архитектура итоговой реализации

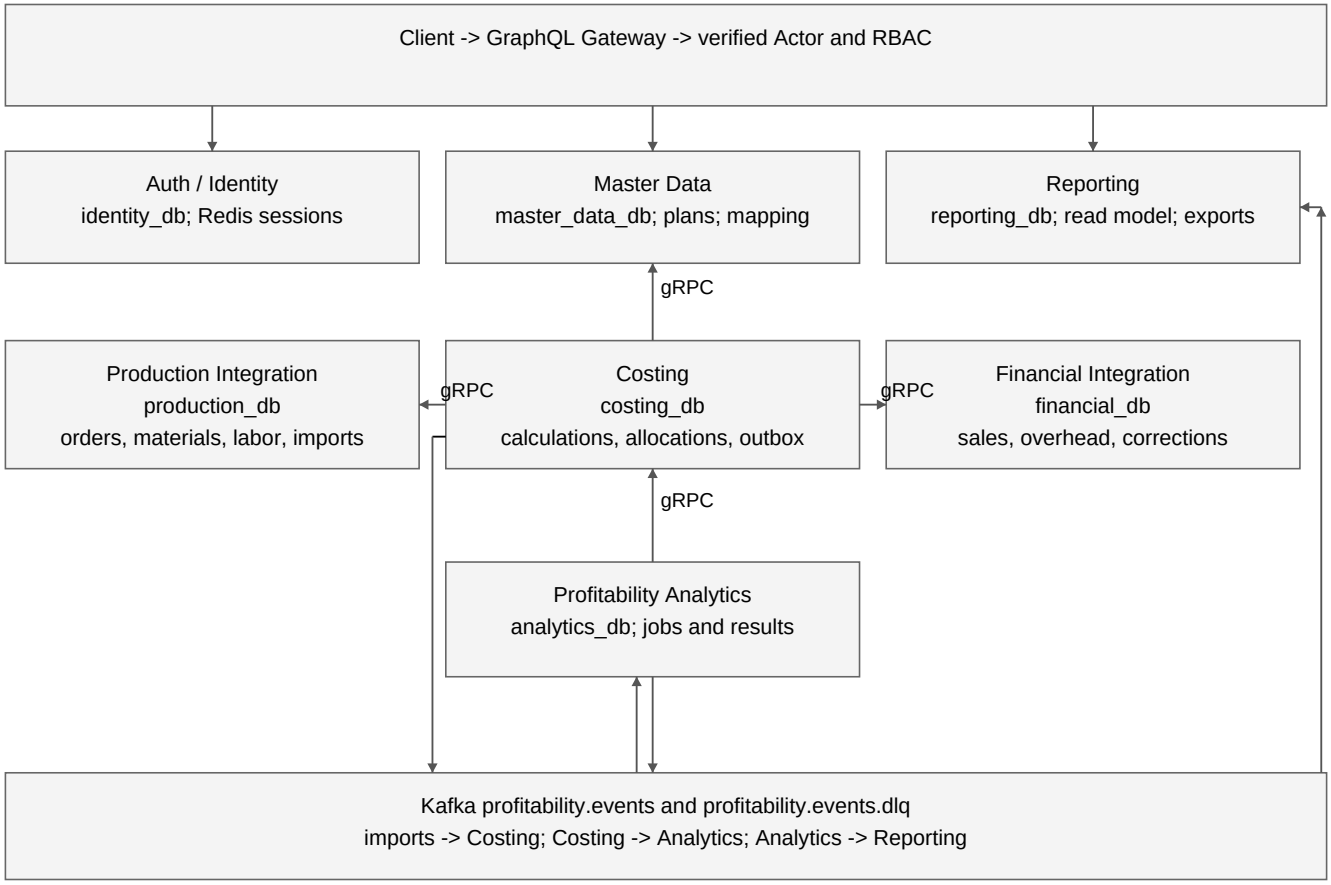


Рисунок. Сервисы, отдельные базы данных и асинхронное обновление Reporting

8 Проверка и запуск

Регрессионные тесты проверяют сохранённые факты и суммы после импорта, откат ошибочного заказа, повтор без дубликатов, готовые файлы трёх форматов, отказ файловой системы, пустую группу в списке и сводке, сумму строк OVERHEAD_ALLOCATION и сохранённые версии источников. Вся итоговая реализация проверяется общим набором тестов ЛР7.

```
pip install -r requirements.txt
python -m pytest -q
# Из каталога итогового проекта:
docker compose build
docker compose run --rm auth python -m app.scripts.seed
docker compose up -d --wait
```

Для Compose требуется INTERNAL_JWT_SECRET в .env. GraphQL доступен на http://127.0.0.1:8000/graphql. Примеры входных файлов лежат в examples/imports, готовые выгрузки — в runtime-data/exports. Seed предназначен для начальной демонстрационной базы и очищает существующие демонстрационные данные.

Заключение

Операции импорта и экспорта доведены до сохранённого результата. Повтор импорта обрабатывает исправленный источник без дублирования фактов; фильтрация пустого набора не расширяет выборку; распределения накладных расходов сохраняются и доступны для проверки. GraphQL остаётся внешним прикладным контрактом, а бизнес-правила выполняются внутри сервисов.